PLC Implementation in GX Works2

Adaptive Demand Allocation Architecture (ADA) — Programming Reference

---

Ladder Logic, Tag Nomenclature, and an Experimental Structured Text (ST) Translation

Author	Sambridda Ranabhat
Organisation	Mechatronics Engineering Pvt. Ltd. (MEPL)
Location	Kathmandu, Nepal
Date	July 19, 2026
Status	Programming Reference — For Review

This document is a programming reference companion to the *Thermofluid Analysis of Architecture II* and its PLC Implementation section: nomenclature, a consolidated tag/ID lookup, every ladder logic block in one place, and an experimental Structured Text translation flagged accordingly. Design rationale, derivations, and open items live in the parent architecture document, not here.

1 Nomenclature

1.1 Instruction Set (FX-Series-Style, via GX Works2)

Table 1: Ladder instruction abbreviations used throughout this document.

Instruction	Meaning
LD / LDI	Load contact (normally-open / normally-closed) – starts a rung
AND / ANI	Series AND (normally-open / normally-closed)
OR / ORI	Parallel OR (normally-open / normally-closed)
OUT	Drive a coil/flag
SET / RST	Latch ON / latch OFF (retains state until explicitly reset)
MOV	Move/copy a value into a register
MUL / DIV / ADD / SUB	16-bit arithmetic
DMUL / DADD / DDIV	32-bit (double-word) arithmetic – required when a value exceeds the 16-bit signed range (−32768 to 32767)
INC	Increment a register by 1
CMP	Compare, sets a 3-bit result (>, =, <)
PLS	One-scan pulse on a rising edge
FROM / TO	Read from / write to an intelligent module's buffer memory
FOR / NEXT	Repeat a block of rungs a fixed number of times (FNC08/09) – the loop body must step V/Z itself each pass; there is no auto-increment. See Section 3.24.
K	Prefix for a constant (decimal)

1.2 Device Prefixes

Table 2: PLC device types.

Prefix	Device type
X	Physical digital input
Y	Physical digital output
M	Internal relay/flag (not a physical point)
D	Data register (16-bit unless part of a 32-bit pair)
T	Timer
V, Z	Index register (16-bit) – modifies another device's address at runtime, e.g. `D700Z` means "D700 + current value of Z." See Section 2.6.
AI-#	Analog input channel (placeholder label, pending real tag numbers)

1.3 Process and Architecture Terms

Table 3: ADA-specific abbreviations.

Term	Meaning
ADA	Adaptive Demand Allocation Architecture
P1 / P2 / P3	Priority tiers: T_5 trim-heat / PO tank maintenance / T_{10} palm oil melt
LSM / HSM / FSM	Trunk flow states: Low/High/Full Supply Mode – 1, 2, or 3 destinations served simultaneously
PO tank	Palm olein tank – whichever of TA/TB/TC is currently designated
S_{casual}, $S_{\text{immediate}}$	Tank scoring functions (volume-weighted / temperature-weighted)
SV	Solenoid valve (water routing)
PBV	Pneumatic ball valve (oil dispense)
HE1–3	Auxiliary immersion heating elements, 12 kW each
AODD	Air-operated double-diaphragm pump ($T_{10} \rightarrow T_5$ transfer)
VFD-01 / VFD-02	Trunk circulation drive / oil distribution drive
T_f	Dynamic upper thermal boundary (T_5 safety cutoff)
Eco/Efficiency/Standard/Full Throttle	P3 operating-posture modes, driven by T_5's level (α_S)
E_{total}	Sum of remaining energy demand across all currently- served branches ($E_\alpha + E_5 + E_{C^*}$), Section 3.11
P_{base}, E_{req}	E_{total} converted to an average power over the planning window, and the throttled power request derived from it (Dynamic Power Evaluation, Section 3.13)
Planning window	5 min **(proposed, not user-confirmed)** – the time basis used to convert remaining energy to an average power figure; see the warn-box in Section 3.11

2 Tag ID Tables

2.1 Digital I/O (Confirmed)

Table 4: Digital inputs.

Tag	Correspondence	Notes
X0	E-STOP	NC, hardwired safety assumed upstream
X1	Automation Enable/Disable	
X2	Circulation Pump (manual)	
X3	Oil Distribution Pump (manual)	
X4	Jacket Pump (manual)	Unused – Y4 runs continuously, see Section 3.4 note
X5	AODD Pump (manual)	
X6	Float – HEA, NC	Warning only
X7	Float – SOL, NC	Warning only
X10	Loop Exit	Manual-mode only
X_Confirm	PO tank designation confirm	Momentary, HMI or push-button
X_EStopReset	E-STOP reset	Momentary, must follow physical E-STOP clear
X_AlarmReset	Master alarm acknowledge	Momentary, HMI or push-button

X_Confirm, X_EStopReset, and X_AlarmReset are used by name in the ladder logic but have not been assigned real numeric addresses – add them here once confirmed, keeping the naming consistent with X0–X10 above.

Table 5: Digital outputs.

Tag	Correspondence	Notes
Y0	E-STOPPED lamp	
Y1	Automation Enabled lamp	
Y2	Circulation Pump – HSM	
Y3	Oil Distribution Pump active	
Y4	Jacket Pump active	Always on with M2_SystemOK – no manual/auto split
Y5	AODD Pump active	
Y6	Circulation Pump – LSM	
Y7	Heating Element 1	12 kW, now gated on M2_SystemOK (Section 3.14)
Y10	Heating Element 2	12 kW, gated on M2_SystemOK
Y11–Y14	SV100–SV103	T5/TA/TB/TC branch valves
Y15/Y16	SV104/SV105	Solar selector: through/bypass
Y17	Circulation Pump – FSM	
Y20	Heating Element 3	12 kW, gated on M2_SystemOK
Y21–Y24	PBV100–PBV103	T5/TA/TB/TC oil valves
Y25	Solar Cycle Active lamp	

Y4 (Jacket Pump active) intentionally does not follow the Manual/Automatic Gating Pattern used by the other pumps – it runs continuously whenever the system is not E-STOPped, with no manual/auto split. X4 is therefore intentionally unused.

2.2 Analog Inputs (Channel Count Confirmed, Tags Pending)

Table 6: Analog inputs.

Channel	Correspondence	Register
AI-1–4	Level: T5/TA/TB/TC (4-20mA ultrasonic)	$\alpha_S, \alpha_{10A,B,C}$
AI-5–8	Temp: T5/TA/TB/TC (4-20mA Pt100)	$\theta_S, \theta_{10A,B,C}$
AI-9/10	Temp: Solar loop / Thermal battery	$\theta_{\text{Sol}}, \theta_H$
AI-11	Flow: Trunk/outlet (4-20mA, confirmed)	Flow rate

2.3 Data Registers (D) by Function

Table 7: Register groups – see the full architecture document for exact per-register purpose within each group.

Group	Range/registers
AI scaling	D0, D2, D4 (per-channel, repeated pattern)
PO designation	D50 (pending), D51 (active)
T_{water}	D200–D204, D_Ptotal, D_Twater_target
Mode	D_Mode, D_AlphaS
Suspicion	D_Suspicion
E_{total} **(now computed, not just consumed)**	D700–D744, D_EalphaTA/TB/TC, D_E5, D_E5sensible, D_E5loss, D_EC_star, D_Etotal – Section 3.11. **An indexed alternative that collapses most of this to shared scratch plus one accumulator is given in Section 3.24.**
Dynamic power	D_PbaseW, D_Pbase, D_Twater, D_TempDiff, D_TempCalc, D_Ereq_kW – Section 3.13
Aux staging	D_AuxStage, D_Ereq_kW
Scoring	D410–D418, D_ScoreCasualXX, D_ScoreImmXX, D_EligCasualXX, D_MaxScore, D_MaxID, D_RankedTankTemp – **not a candidate for indexing**: the ranking/tie-break logic needs all three tanks' scores live simultaneously, so this stays three named registers regardless (see the note in Section 3.24)
Mass conversion	D510, D520, D_MassT5/TA/TB/TC, D_HeightT5/TA/TB/TC
T_f	D330–D371, D_ThetaH, D_ThetaS, D_Mp (**now sourced**, Section 3.12), D_Tf
Arbiter	D_NeedCount, D_ServeCount
Fault detection	D600–D622, D_DeltaThetaXX, D_MaxHeatRateXX, D_MaxCoolRateXX, D_DeltaMassXX – **an indexed alternative is given in Section 3.24**, collapsing the triplicated scratch chain to one shared copy; the per-tank _Prev registers stay three-wide (persistent state), reorganised as Z-indexed arrays rather than eliminated

D_Mp **is now resolved.** It is computed, not entered or measured directly: T5's remaining capacity to reach 100% full, derived entirely from T5's own live mass (Section 3.12). This was an open item in the previous version of this document and is now closed.

2.4 Working Flags (M) by Function

Table 8: Flag groups.

Group	Flags
Safety/mode	M1_ESTOP, M2_SystemOK, M3_AutoEnabled
PO designation	M0–M3
Demand	M_P1Demand, M_P2Demand, M_D_T5/TA/TB/TC
Arbiter	M_Serve_T5/TA/TB/TC, M_Challenger
T_f/P1	M_TfInterlock, M4_ThrottleOrDivert, M_BatchRelease
P2	M_Overrun_Pulse, M_PromptActive, M_HMIPrompt, M_Alarm, M_DisableFeed
Oil distribution	M_DemandX/Y/Z, M_ServeX/Y/Z
Fault detection	M_FaultHeatingXX, M_FaultCoolingXX, M_FaultLevelXX, M_HighLimitXX, M_TAActiveTransfer
Fast override (proposed, T5 only, see Section 3.13)	M_FastOverride_T5

Naming collision risk, flagged not fixed. Oil Distribution's `M_ServeZ` (the third oil-destination flag, named for the X/Y/Z demand triple) is spelled identically to a bare `Z` index-register reference once Z-indexing is introduced (Section 3.24). The two are unrelated – `M_ServeZ` is always the full device name, never followed directly by another device to modify – but this is the same class of risk as the `M1_ESTOP`/`M1` collision fixed elsewhere in this revision. Recommend renaming `M_DemandZ`/`M_ServeZ` (e.g. to `M_DemandC`/`M_ServeC`) if Z-indexing is adopted, to keep the two visually distinct at address-assignment time. Not renamed here to avoid touching the already-established Oil Distribution block outside the scope of this pass.

2.5 Timers (T)

Table 9: Timers.

Timer	Purpose
T0	10s refresh oscillator (self-resetting)
T5	Suspicion meter 5-min no-response auto-increment

2.6 Index Registers (V, Z)

Table 10: Index register usage.

Register	Purpose
Z0	Tank-loop index (TA=0/TB=1/TC=2), stride K1 – Section 3.24
Z1–Z7	Unused, reserved
V0–V7	Unused, reserved

This uses 1 of the 16 available index-register points (V0–V7/Z0–Z7) on every L01S model per the hardware table – well within budget regardless of which model is ultimately chosen. FOR/NEXT and Z-indexed device addressing are standard FX-series features, but – same caveat as ST support (Section 4) – availability on the actual Coolmay L01S has not been separately confirmed. Confirm both together before committing hardware design to either optimization.

Every address above is an illustrative placeholder, coordinated only to avoid collisions with itself – confirm a real, complete address map before programming.

3 Ladder Logic – Full Listing

3.1 First-Scan Initialization

```
LD   M8002
RST  M_Alarm
RST  M_HighLimitTA
RST  M_HighLimitTB
RST  M_HighLimitTC
RST  M_FaultHeatingTA  RST M_FaultCoolingTA  RST M_FaultLevelTA
RST  M_FaultHeatingTB  RST M_FaultCoolingTB  RST M_FaultLevelTB
RST  M_FaultHeatingTC  RST M_FaultCoolingTC  RST M_FaultLevelTC
MOV  K0 D_Suspicion
MOV  K0 D_AuxStage
MOV  K0 D_Ereq_kW
MOV  K0 D_NeedCount
MOV  K0 D_ServeCount
```

3.2 AI Channel Read/Scale

```
FROM K0 K10 D0 K1
MUL  D0  K100  D2
DIV  D2  K4000 D4
```

3.3 Master Safety and Mode Control

Naming collision risk, fixed this revision. The previous version used `M1_ESTOP` alongside the bare `M1` (PO designation = TA) elsewhere in the program – two unrelated bits with dangerously similar names. E-STOP's latch flag is renamed `M_ESTOP` throughout this revision to remove the risk of the two being confused at address-assignment time.

```
LDI X0
SET M_ESTOP
LD  X_EStopReset
RST M_ESTOP
LD  M_ESTOP
OUT Y0
LDI M_ESTOP
OUT M2_SystemOK

LD  X1
OUT Y1
LD  X1
OUT M3_AutoEnabled
```

3.4 Manual/Automatic Gating Pattern (repeat per pump/valve)

```
LD  M2_SystemOK
ANI M3_AutoEnabled
AND X2
OUT [pump output]

LD  M2_SystemOK
AND M3_AutoEnabled
AND [arbiter / need-array condition]
OUT [pump output]
```

3.5 Jacket Pump

```
LD  M2_SystemOK
OUT Y4
```

3.6 PO Tank Designation

```
LD  X_Confirm
PLS M100
LD  M100
MOV D50  D51

LD= D51 K0   OUT M0
LD= D51 K1   OUT M1
LD= D51 K2   OUT M2
LD= D51 K3   OUT M3
```

3.7 10-Second Refresh Oscillator

```
LDI T0
OUT T0 K100
LD  T0
OUT M10_Refresh
```

3.8 T_{water} Target

```
LD  M10_Refresh
MUL D_Ptotal K300  D200
DIV D200     K6160 D202
ADD D202     K400  D204

LD< D204 K400
MOV K400 D204
LD> D204 K700
MOV K700 D204
MOV D204 D_Twater_target
```

3.9 Mode Register

```
LD= D_Mode K0   AND< D_AlphaS K85   MOV K1 D_Mode
LD= D_Mode K1   AND>= D_AlphaS K90  MOV K0 D_Mode
LD= D_Mode K1   AND< D_AlphaS K65   MOV K2 D_Mode
LD= D_Mode K2   AND>= D_AlphaS K70  MOV K1 D_Mode
LD= D_Mode K2   AND< D_AlphaS K45   MOV K3 D_Mode
LD= D_Mode K3   AND>= D_AlphaS K50  MOV K2 D_Mode
```

3.10 Suspicion Score

```
LD  M_Overrun_Pulse
INC D_Suspicion
LD> D_Suspicion K3
MOV K3 D_Suspicion

LD= D_Suspicion K1  OUT M_HMIPrompt
LD= D_Suspicion K2  OUT M_Alarm
LD= D_Suspicion K3  OUT M_DisableFeed
LD= D_Suspicion K3  OUT M_Alarm

LD  M_PromptActive
OUT T5  K3000
LD  T5
INC D_Suspicion
RST M_PromptActive

LD  M_DailyReset_Pulse
MOV K0 D_Suspicion
```

3.11 E_{total} Calculation

Planning window = 5 minutes, proposed, not user-confirmed. Used to convert the loss terms below (Watts) into energy (Joules) over a forward-looking window, and again in Dynamic Power Evaluation (Section 3.13) to convert total remaining energy back into an average power figure. A shorter window (closer to the existing 10s master refresh) responds faster to real changes but is noisier; a much longer window (e.g. 1 hour, considered and rejected during development) risks holding a stale, too-high setpoint for longer than some real duty cycles (the PO tank's own dispense duty is ≈42 min) – 5 minutes was chosen as a middle point, not derived from any established figure elsewhere in this project.

Corrects a units error in the original E_5/E_{C^*} derivation: standing loss ($UA\Delta T$) is a *power* (W), but was previously added directly to a sensible-heat *energy* term (J) with no time conversion between them. Every loss term below is now multiplied by the planning window (K300, seconds) before being summed.

An indexed alternative to the TA/TB/TC portion of this block – see Section 3.24 – collapses most of the scratch and result registers below into shared scratch plus one running accumulator. The baseline (fully written-out, one rung group per tank) version is kept below as the confirmed-correct reference; the indexed version is proposed and unverified pending FOR/NEXT and Z-indexing confirmation on the actual target hardware.

```
// E_alpha, tank TA -- only counts if TA is being served AND is a melt
// duty (not the PO tank). Piecewise per the base architecture: below
// 36C still needs sensible-to-melt + latent; 36-40C is liquid sensible
// only. Repeat identically for TB, TC.
LD  M_Serve_TA
ANI M1
AND<= D_ThetaTA K360
SUB  K360 D_ThetaTA D700          // (36.0 - ThetaTA), tenths degC
DMUL D700 K200 D702                 // x Cp_solid(2000) -- sensible term
DADD D702 K158400000 D704            // + latent+loss allowance (scaled)
DMUL D704 D_MassTA D_EalphaTA        // x mass -> Joules

LD  M_Serve_TA
ANI M1
AND> D_ThetaTA K360
AND<= D_ThetaTA K400
SUB  K400 D_ThetaTA D710
DMUL D710 K210 D712                  // Cp_liquid(2100)
DMUL D712 D_MassTA D_EalphaTA

// E_5 -- sensible term + loss term (now correctly window-scaled)
SUB  D_TF D_ThetaS D720
DMUL D720 K210 D722
DMUL D722 D_Mass5 D_E5sensible        // Joules

SUB  D_ThetaS K181 D724               // (ThetaS - ambient 18.1C)
DMUL K978 D724 D726                    // U5*A5, scaled x1000 -- Watts x1000
DMUL D726 K300 D728                     // x window(s) -> Joules x1000
DDIV D728 K1000 D_E5loss

DADD D_E5sensible D_E5loss D_E5

// E_C_star (PO tank) -- identical pattern to E_5, own U*A and mass
SUB  K300 D_ThetaC D730                // target 30.0C - ThetaC
DMUL D730 K210 D732
DMUL D732 D_MassPO D_ECsensible

SUB  D_ThetaC K181 D734
DMUL K5908 D734 D736                    // U_PO*A_PO, scaled x1000
DMUL D736 K300 D738
DDIV D738 K1000 D_ECloss

DADD D_ECsensible D_ECloss D_EC_star

// Sum only branches actually being served this cycle
DADD D_EalphaTA D_EalphaTB D740
DADD D740        D_EalphaTC D742
DADD D742        D_E5       D744
DADD D744        D_EC_star  D_Etotal
```

Every intermediate above needs 32-bit (`DMUL/DADD/DDIV`) – same overflow risk as the T_f block, for the same reason (large constants, kg-scale masses). `K978` (T5's UA, scaled) and `K5908` (PO tank's UA, scaled) are carried over from the architecture document's established loss coefficients – confirm scaling factors match before use. D_ThetaC/D_MassPO assume the currently- designated PO tank's readings are copied into dedicated registers elsewhere (not shown) rather than re-resolving M1/M2/M3 inline here.

3.12 T_f Thermal Blending

```
// m_p: T5's remaining capacity to reach 100% full, from T5's own
// live mass -- NOT read from a T10 tank.
SUB K4871 D_MassT5 D_Mp

MOV D_ThetaH D330   MOV K0 D331
MOV D_ThetaS D332   MOV K0 D333
MOV D_Mp     D334   MOV K0 D335

DMUL K75808 D330 D340
DMUL D334   K2000 D344
DMUL D344   D332  D350

DADD D340   D350  D360
DADD K75808 D344  D362

DDIV D360 D362 D370
MOV  D370 D_Tf

LD>= D_Tf K550
OUT  M_TfInterlock
LD   M_TfInterlock
OUT  M4_ThrottleOrDivert
```

`K4871` is T5's full-tank mass at 2000 mm (2.4353 kg/mm×2000), matching the height-to-mass constant established elsewhere in this document.

3.13 Dynamic Power Evaluation

```
// E_total -> average power over the planning window (Watts), then kW
LD  M10_Refresh
DDIV D_Etotal K300  D_PbaseW      // Joules / 300s = Watts
DDIV D_PbaseW K1000 D_Pbase        // Watts / 1000 = kW

// Condition 1: measured water temp < 40.0C (full power)
LD  M10_Refresh
AND< D_Twater K400
MOV D_Pbase D_Ereq_kW

// Condition 2: measured water temp >= 70.0C (zero power -- safety taper)
LD  M10_Refresh
AND>= D_Twater K700
MOV K0 D_Ereq_kW

// Condition 3: 40.0C <= measured water temp < 70.0C (proportional taper)
// -- unchanged from the original design; D_Twater here is MEASURED,
// not the target, so this is a protective ceiling-approach taper,
// independent of demand, layered on top of D_Pbase.
LD  M10_Refresh
AND>= D_Twater K400
AND< D_Twater K700
SUB K700 D_Twater D_TempDiff
MUL D_Pbase D_TempDiff D_TempCalc
DIV D_TempCalc K300 D_Ereq_kW
```

Fast override, proposed and only sketched for T5 – not a complete implementation. Purpose: prevent a stale (up to 5-min-old) high E_{req} from continuing to command power once a served tank has demonstrably already reached its own target, rather than waiting for the next planning-window refresh.

```
LD  M10_Refresh
AND< D_ThetaS D_Twater_target
OUT M_FastOverride_T5

LD  M_FastOverride_T5
MOV K0 D_Ereq_kW
```

This checks only T5. Extending to the PO tank and T10 melt duties (and combining via OR before the final `MOV`) is the same pattern but was not worked out per-duty here – treat this as a direction, not a finished block.

3.14 Auxiliary Stage Register

```
LD= D_AuxStage K0  AND> D_Ereq_kW K0   MOV K1 D_AuxStage
LD= D_AuxStage K1  AND<= D_Ereq_kW K2  MOV K0 D_AuxStage
LD= D_AuxStage K1  AND>= D_Ereq_kW K14 MOV K2 D_AuxStage
LD= D_AuxStage K2  AND< D_Ereq_kW K10  MOV K1 D_AuxStage
LD= D_AuxStage K2  AND>= D_Ereq_kW K26 MOV K3 D_AuxStage
```

```
LD= D_AuxStage K3  AND< D_Ereq_kW K22  MOV K2 D_AuxStage

// Now gated on M2_SystemOK -- was missing in the previous version
LD  M2_SystemOK
AND>= D_AuxStage K1
OUT Y7

LD  M2_SystemOK
AND>= D_AuxStage K2
OUT Y10

LD  M2_SystemOK
AND>= D_AuxStage K3
OUT Y20
```

3.15 Tank Scoring (worked for TA; repeat for TB, TC)

```
MUL D_ThetaTA K100 D410
DIV D410      K700 D_TnormTA

MUL D_AlphaTA K6 D412
MUL D_TnormTA K4 D414
ADD D412 D414 D_ScoreCasualTA

MUL D_AlphaTA K4 D416
MUL D_TnormTA K6 D418
ADD D416 D418 D_ScoreImmTA
```

3.16 PO Exclusion and Ranking

```
LD  M1  MOV K-1 D_EligCasualTA
LDI M1  MOV D_ScoreCasualTA D_EligCasualTA
LD  M2  MOV K-1 D_EligCasualTB
LDI M2  MOV D_ScoreCasualTB D_EligCasualTB
LD  M3  MOV K-1 D_EligCasualTC
LDI M3  MOV D_ScoreCasualTC D_EligCasualTC

MOV K1 D_MaxID
MOV D_EligCasualTA D_MaxScore

LD> D_EligCasualTB D_MaxScore
OUT M_Challenger
LD M_Challenger  MOV D_EligCasualTB D_MaxScore
LD M_Challenger  MOV K2 D_MaxID

LD> D_EligCasualTC D_MaxScore
OUT M_Challenger
LD M_Challenger  MOV D_EligCasualTC D_MaxScore
LD M_Challenger  MOV K3 D_MaxID
```

```
LD= D_MaxID K1  MOV D_ThetaTA D_RankedTankTemp
LD= D_MaxID K2  MOV D_ThetaTB D_RankedTankTemp
LD= D_MaxID K3  MOV D_ThetaTC D_RankedTankTemp
```

3.17 Height-to-Mass Conversion

```
SUB K2000 D_DistT5 D_HeightT5
DMUL D_HeightT5 K24353 D510
DDIV D510       K10000 D_MassT5
// T10 tanks: SUB K2200 ..., DMUL ... K40455, same pattern
```

3.18 P1 – T5 Block

```
LD<  D_ThetaS K400
OUT  M_P1Demand

LD   M_TfInterlock
OUT  M4_ThrottleOrDivert

LD<   D_AlphaS K50
AND>= D_RankedTankTemp K360
OUT   M_BatchRelease
LD    M_BatchRelease
OUT   Y5
```

> Y11 is intentionally *not* driven here – it is driven once, from `M_Serve_T5`, in the Need-Array Arbiter below. This resolves the double-drive issue flagged in the previous version of this document.

3.19 P2 – PO Tank Block

```
LD  D_DispensedVol
AND>= D_DispensedVol D_2xNd
OUT M_Overrun_Pulse

LD  M1  OR M2  OR M3
AND [dispense active, not yet "process complete"]
OUT M_P2Demand
```

3.20 Need-Array Arbiter

```
LD  M_P1Demand              OUT M_D_T5
LD  M1  AND M_P2Demand      OUT M_D_TA_asPO
LD  M2  AND M_P2Demand      OUT M_D_TB_asPO
LD  M3  AND M_P2Demand      OUT M_D_TC_asPO

LDI M1  AND< D_ThetaTA K360  OUT M_D_TA_asMelt
LDI M2  AND< D_ThetaTB K360  OUT M_D_TB_asMelt
LDI M3  AND< D_ThetaTC K360  OUT M_D_TC_asMelt

LD M_D_TA_asPO  OR M_D_TA_asMelt  OUT M_D_TA
LD M_D_TB_asPO  OR M_D_TB_asMelt  OUT M_D_TB
LD M_D_TC_asPO  OR M_D_TC_asMelt  OUT M_D_TC

MOV K0 D_NeedCount
LD M_D_T5  INC D_NeedCount
LD M_D_TA  INC D_NeedCount
LD M_D_TB  INC D_NeedCount
LD M_D_TC  INC D_NeedCount

LD M_D_T5  OUT M_Serve_T5
LD M_D_TA  OUT M_Serve_TA
LD M_D_TB  OUT M_Serve_TB
LD M_D_TC  OUT M_Serve_TC

LD= D_NeedCount K4  ANI M0  AND M1
LD< D_ScoreImmTB D_ScoreImmTC   RST M_Serve_TB
LD= D_NeedCount K4  ANI M0  AND M1
LD>= D_ScoreImmTB D_ScoreImmTC  RST M_Serve_TC

LD= D_NeedCount K4  ANI M0  AND M2
LD< D_ScoreImmTA D_ScoreImmTC   RST M_Serve_TA
LD= D_NeedCount K4  ANI M0  AND M2
LD>= D_ScoreImmTA D_ScoreImmTC  RST M_Serve_TC

LD= D_NeedCount K4  ANI M0  AND M3
LD< D_ScoreImmTA D_ScoreImmTB   RST M_Serve_TA
LD= D_NeedCount K4  ANI M0  AND M3
LD>= D_ScoreImmTA D_ScoreImmTB  RST M_Serve_TB

LD= D_NeedCount K4  AND M0
AND< D_ScoreImmTA D_ScoreImmTB  AND< D_ScoreImmTA D_ScoreImmTC
RST M_Serve_TA
LD= D_NeedCount K4  AND M0
AND< D_ScoreImmTB D_ScoreImmTA  AND< D_ScoreImmTB D_ScoreImmTC
RST M_Serve_TB
LD= D_NeedCount K4  AND M0
AND< D_ScoreImmTC D_ScoreImmTA  AND< D_ScoreImmTC D_ScoreImmTB
RST M_Serve_TC
```

```
MOV K0 D_ServeCount
LD M_Serve_T5  INC D_ServeCount
LD M_Serve_TA  INC D_ServeCount
LD M_Serve_TB  INC D_ServeCount
LD M_Serve_TC  INC D_ServeCount

LD M2_SystemOK  AND M3_AutoEnabled  AND= D_ServeCount K1  OUT Y6
LD M2_SystemOK  AND M3_AutoEnabled  AND= D_ServeCount K2  OUT Y2
LD M2_SystemOK  AND M3_AutoEnabled  AND= D_ServeCount K3  OUT Y17

LD M_Serve_T5  OUT Y11    LD M_Serve_TA  OUT Y12
LD M_Serve_TB  OUT Y13    LD M_Serve_TC  OUT Y14

LD  M2_SystemOK
ANI M3_AutoEnabled
AND X2
OUT Y2
```

3.21 Solar and Auxiliary Gating

```
LD  M3_AutoEnabled
AND> D_ThetaSol D_Twater_target
OUT Y15

LDI M3_AutoEnabled
AND X10
OUT Y15

LD  M3_AutoEnabled
AND<= D_ThetaSol D_Twater_target
OUT Y16

LDI M3_AutoEnabled
ANI X10
OUT Y16

LD  Y15
OUT Y25
```

3.22 Oil Distribution

```
LD  M_DemandX
OUT M_ServeX

LDI M_DemandX  AND M_DemandY
OUT M_ServeY

LDI M_DemandX  ANI M_DemandY  AND M_DemandZ
OUT M_ServeZ
```

```
LD M_ServeX  OUT Y22
LD M_ServeY  OUT Y21
LD M_ServeY  OUT Y24
LD M_ServeZ  OUT Y21
LD M_ServeZ  OUT Y23

LD M_ServeX  OR M_ServeY  OR M_ServeZ
OUT Y3
```

Open item carried forward: `M_ServeX` opens no T5 outlet valve (Y21) while `M_ServeY`/`M_ServeZ` both do – confirm whether this asymmetry is intentional before this goes to hardware.

3.23 Fault Detection

An indexed alternative to this block – see Section 3.24 – collapses the triplicated scratch chain below to one shared copy. The baseline version is kept below as the confirmed-correct reference.

```
// --- TANK A ---
LD  M10_Refresh
SUB D_ThetaTA D_ThetaTA_Prev D_DeltaThetaTA
LD  M10_Refresh
MOV D_ThetaTA D_ThetaTA_Prev

DMUL K5011  K100  D600
DDIV D600   D_MassTA D602
DDIV D602   K2000   D_MaxHeatRateTA

SUB  D_ThetaTA K181 D610
DMUL K668  D610  D620
DDIV D620  D_MassTA D622
DDIV D622  K2000    D_MaxCoolRateTA

LD>  D_DeltaThetaTA D_MaxHeatRateTA
OUT  M_FaultHeatingTA
LD<  D_DeltaThetaTA K0
AND> D_DeltaThetaTA D_MaxCoolRateTA
OUT  M_FaultCoolingTA

LD  M10_Refresh
SUB D_MassTA D_MassTA_Prev D_DeltaMassTA
LD  M10_Refresh
MOV D_MassTA D_MassTA_Prev

LD  M_ServeTA_AODD
OR  M_ServeX  OR M_ServeY  OR M_ServeZ
OUT M_TAActiveTransfer
```

```
LDI M_TAActiveTransfer
AND<> D_DeltaMassTA K0
OUT M_FaultLevelTA

LD>  D_ThetaTA K750
OUT  M_HighLimitTA
// (repeat identically for TB, TC)

LD   M_HighLimitTA
OR   M_HighLimitTB
OR   M_HighLimitTC
SET  M_Alarm

LD   M_Alarm
RST  Y7
RST  Y10
RST  Y20
```

3.24 Indexed Tank Loop (Proposed) – Reduces D Register Count

Proposed and unverified, same caveat family as Section 4 (ST). This subsection is an *alternative* implementation of the TA/TB/TC portions of E_{total} Calculation (Section 3.11) and Fault Detection (Section 3.23), using `FOR`/`NEXT` (FNC08/09) and Z0-indexed device addressing to loop over the three tanks instead of writing the same rung group out three times. It does not replace the baseline versions above – those remain the confirmed-correct reference – and should not be relied on until every point below is confirmed on the actual target hardware:

- `FOR`/`NEXT` and Z-register device modification are standard FX-series features, but availability on the Coolmay L01S specifically has not been established (see Section 2.6).
- **This requires a register-layout change, not just new ladder.** *D_ThetaT_Arr*, *D_MassT_Arr*, *D_ThetaTPrev_Arr*, *D_MassTPrev_Arr* below must actually be laid out as three contiguous single-word slots (index 0=TA, 1=TB, 2=TC) for `DeviceZ` addressing to reach the right tank – they are not contiguous today, since every other section of this document addresses tanks by name (*D_ThetaTA*, *D_ThetaTB*, ...). Likewise *M_ServeT_Arr*/*M_FaultHeatingT_Arr*/etc. need the corresponding M-flags laid out as 3-point blocks. Real base addresses are left symbolic here, consistent with the rest of this document's placeholder addressing.
- **Stride is K1 (single word) throughout below.** Both Theta (tenths of °C, max seen elsewhere in this document is 750) and Mass (max seen is T5's own full-tank constant, K4871) comfortably fit a single 16-bit word despite being produced via 32-bit (`DMUL`/`DDIV`) arithmetic upstream – confirm this against real tank capacities before relying on it, particularly for TA/TB/TC if their full-tank mass constants turn out larger than T5's.
- `DeviceZ` notation below means "the device address plus the current value of Z0" (FX syntax, e.g. `D700Z` = D700 + Z0) – not a new device family.

```
// ------------------------------------------------------------
// E_alpha loop -- replaces the triplicated TA/TB/TC rung groups in
// Section 3.11 with one loop and a running accumulator. E_5 and
// E_C_star (T5 and the PO tank -- not part of the TA/TB/TC loop)
// are unchanged from Section 3.11 and still feed into D_Etotal below.
// ------------------------------------------------------------
MOV K0 D_EalphaSum
MOV K0 Z0

FOR K3
  // Branch 1: ThetaT[Z] <= 36.0C (sensible-to-melt + latent)
  LD    M_ServeT_ArrZ
  ANI   M_PODesig_ArrZ            // M_PODesig_Arr = M1 (M1/M2/M3 are
  AND<= D_ThetaT_ArrZ K360        // already contiguous -- see note)
  SUB   K360 D_ThetaT_ArrZ D700
  DMUL  D700 K200 D702
  DADD  D702 K158400000 D704
```

```
  DMUL  D704 D_MassT_ArrZ D706
  DADD  D_EalphaSum D706 D_EalphaSum

  // Branch 2: 36.0C < ThetaT[Z] <= 40.0C (liquid sensible only)
  LD    M_ServeT_ArrZ
  ANI   M_PODesig_ArrZ
  AND>  D_ThetaT_ArrZ K360
  AND<= D_ThetaT_ArrZ K400
  SUB   K400 D_ThetaT_ArrZ D710
  DMUL  D710 K210 D712
  DMUL  D712 D_MassT_ArrZ D706
  DADD  D_EalphaSum D706 D_EalphaSum

  INC Z0
NEXT

// D_EalphaSum now holds EalphaTA+EalphaTB+EalphaTC directly --
// no separate D740/D742 summing scratch needed.
DADD D_EalphaSum D_E5      D742
DADD D742         D_EC_star D_Etotal
```

M_PODesig_ArrZ needs no new registers: M1/M2/M3 (PO designation = TA/TB/TC) are already contiguous in the Working Flags table, so `M1Z` reaches M1/M2/M3 directly for Z=0/1/2. *M_ServeT_ArrZ* is *not* currently contiguous – *M_Serve_TA*/`TB`/`TC` would need to be re-addressed as a 3-point block (Need-Array Arbiter, Section 3.20) for this to work as written; otherwise substitute a short `CMP`/branch on Z to read the correctly-named flag instead of indexing.

```
// ------------------------------------------------------------
// Fault Detection loop -- replaces the triplicated TA/TB/TC scratch
// chain in Section 3.23. D_ThetaTPrev_Arr/D_MassTPrev_Arr remain
// three-wide (persistent state cannot be shared across tanks) but are
// now one indexed array each instead of three named registers;
// D600/602/610/620/622/D_DeltaTheta/D_DeltaMass become shared scratch,
// reused once per tank per scan instead of tripled.
// ------------------------------------------------------------
MOV K0 Z0

FOR K3
  LD  M10_Refresh
  SUB D_ThetaT_ArrZ D_ThetaTPrev_ArrZ D_DeltaTheta
  LD  M10_Refresh
  MOV D_ThetaT_ArrZ D_ThetaTPrev_ArrZ

  DMUL K5011  K100  D600
  DDIV D600   D_MassT_ArrZ D602
  DDIV D602   K2000        D_MaxHeatRate

  SUB  D_ThetaT_ArrZ K181 D610
  DMUL K668  D610  D620
```

```
  DDIV D620  D_MassT_ArrZ D622
  DDIV D622  K2000        D_MaxCoolRate

  LD>  D_DeltaTheta D_MaxHeatRate
  OUT  M_FaultHeatingT_ArrZ
  LD<  D_DeltaTheta K0
  AND> D_DeltaTheta D_MaxCoolRate
  OUT  M_FaultCoolingT_ArrZ

  LD  M10_Refresh
  SUB D_MassT_ArrZ D_MassTPrev_ArrZ D_DeltaMass
  LD  M10_Refresh
  MOV D_MassT_ArrZ D_MassTPrev_ArrZ

  LD  M_ServeTAODD_ArrZ
  OR  M_ServeX  OR M_ServeY  OR M_ServeZ
  OUT M_TActiveTransfer_ArrZ

  LDI M_TActiveTransfer_ArrZ
  AND<> D_DeltaMass K0
  OUT M_FaultLevelT_ArrZ

  LD>  D_ThetaT_ArrZ K750
  OUT  M_HighLimitT_ArrZ

  INC Z0
NEXT

LD   M_HighLimitT_ArrZ0   // i.e. M_HighLimitTA -- unchanged, OR-chain
OR   M_HighLimitT_ArrZ1   // reads all three post-loop, same as before
OR   M_HighLimitT_ArrZ2
SET  M_Alarm

LD   M_Alarm
RST  Y7
RST  Y10
RST  Y20
```

`M_ServeZ` in the transfer-detection OR chain above is the Oil Distribution "Z-destination" flag (Section 3.22), not a reference to the Z0 index register – see the naming-collision warnbox in Section 2.4. Left as-is here to match the baseline block exactly; worth renaming before this is finalised.

Approximate register savings from this subsection. Counting every named scratch/result register in the two baseline blocks above (Sections 3.11 and 3.23) as if fully written out per tank:

- E_{total} **TA/TB/TC portion**: baseline needs ∼5 scratch registers per tank ×3 (15), plus 3 stored per-tank results ($D_EalphaTA/TB/TC$), plus 3 summing-scratch registers (D740/742/744) – **∼21 total**. Indexed: 6 shared scratch registers (D700/702/704/706/710/712, reused every pass) plus 1 running accumulator ($D_EalphaSum$) – **7 total**, roughly a two-thirds cut.
- **Fault Detection TA/TB/TC portion**: baseline needs ∼9 scratch/result registers per tank (D600/602/610/620/622 plus DeltaTheta/DeltaMass, each per-tank) ×3 (27), plus 2 persistent _Prev registers per tank ×3 (6) – **∼33 total**. Indexed: 9 shared scratch registers (D600/602/610/620/622/$D_MaxHeatRate/D_MaxCoolRate/$ $D_DeltaTheta/D_DeltaMass$) plus 6 persistent registers reorganised into two 3-point arrays ($D_ThetaTPrev_Arr$, $D_MassTPrev_Arr$) – **15 total**, roughly a 55% cut. The persistent arrays don't shrink – three tanks' state genuinely needs three slots – only the transient scratch does.
- **Combined, these two blocks alone drop from ∼54 to ∼22 named D registers** – a savings of ∼32, using only 1 of the 16 available V/Z points (Z0). This was the single largest lever available for staying inside the D0–D127 general-use pool flagged as a risk earlier in this document's review; a full register tally against that 128-point budget is still recommended before finalising, since Scoring (Section 3.15) and several other blocks are *not* candidates for this treatment and still contribute their full baseline count.

3.25 Alarm Acknowledge and Reset

```
LD   X_AlarmReset
ANI  M_HighLimitTA
ANI  M_HighLimitTB
ANI  M_HighLimitTC
RST  M_Alarm
```

4 Structured Text (ST) – EXPERIMENTAL

This section is experimental and unverified. GX Works2's ST support is primarily associated with Mitsubishi's iQ-R/iQ-F hardware family, not the classic FX-series line the ladder logic assumes – ST availability on the Coolmay L01S has not been established. Coverage is partial by design: straightforward blocks are omitted; every block with nontrivial arithmetic or branching is included.

4.1 E_{total} Calculation

```
IF M_Serve_TA AND (NOT M1) THEN
    IF D_ThetaTA <= 360 THEN
        D_EalphaTA := ((360 - D_ThetaTA) * 200 + 158400000) * D_MassTA;
    ELSIF D_ThetaTA <= 400 THEN
        D_EalphaTA := (400 - D_ThetaTA) * 210 * D_MassTA;
    END_IF;
END_IF;
(* repeat identically for TB, TC *)

D_E5sensible := (D_TF - D_ThetaS) * 210 * D_Mass5;
D_E5loss     := (978 * (D_ThetaS - 181) * 300) / 1000;
D_E5         := D_E5sensible + D_E5loss;

D_ECsensible := (300 - D_ThetaC) * 210 * D_MassPO;
D_ECloss     := (5908 * (D_ThetaC - 181) * 300) / 1000;
D_EC_star    := D_ECsensible + D_ECloss;

D_Etotal := D_EalphaTA + D_EalphaTB + D_EalphaTC + D_E5 + D_EC_star;
```

4.2 T_f Thermal Blending

```
D_Mp := 4871 - D_MassT5;   (* T5's own remaining capacity, not read
                              from any T10 tank *)

D_Tf := (75808 * D_ThetaH + D_Mp * 2000 * D_ThetaS) / (75808 + D_Mp * 2000);

M_TfInterlock := (D_Tf >= 550);
M4_ThrottleOrDivert := M_TfInterlock;
```

4.3 Dynamic Power Evaluation

```
IF M10_Refresh THEN
    D_PbaseW := D_Etotal / 300;    (* Joules / 5-min window = Watts *)
    D_Pbase  := D_PbaseW / 1000;    (* -> kW *)

    IF D_Twater < 400 THEN
        D_Ereq_kW := D_Pbase;
    ELSIF D_Twater >= 700 THEN
        D_Ereq_kW := 0;
    ELSE
        D_TempDiff := 700 - D_Twater;
        D_Ereq_kW  := (D_Pbase * D_TempDiff) / 300;
    END_IF;
END_IF;

(* Fast override, T5 only -- proposed, not extended to other duties *)
M_FastOverride_T5 := M10_Refresh AND (D_ThetaS < D_Twater_target);
IF M_FastOverride_T5 THEN
    D_Ereq_kW := 0;
END_IF;
```

4.4 Auxiliary Stage Register

```
CASE D_AuxStage OF
    0: IF D_Ereq_kW > 0 THEN D_AuxStage := 1; END_IF;
    1: IF D_Ereq_kW <= 2 THEN D_AuxStage := 0;
       ELSIF D_Ereq_kW >= 14 THEN D_AuxStage := 2;
       END_IF;
    2: IF D_Ereq_kW < 10 THEN D_AuxStage := 1;
       ELSIF D_Ereq_kW >= 26 THEN D_AuxStage := 3;
       END_IF;
    3: IF D_Ereq_kW < 22 THEN D_AuxStage := 2; END_IF;
END_CASE;

Y7  := M2_SystemOK AND (D_AuxStage >= 1);
Y10 := M2_SystemOK AND (D_AuxStage >= 2);
Y20 := M2_SystemOK AND (D_AuxStage >= 3);
```

4.5 Tank Scoring and Ranking

```
D_TnormTA := (D_ThetaTA * 100) / 700;
D_TnormTB := (D_ThetaTB * 100) / 700;
D_TnormTC := (D_ThetaTC * 100) / 700;

D_ScoreCasualTA := (D_AlphaTA * 6) + (D_TnormTA * 4);
D_ScoreCasualTB := (D_AlphaTB * 6) + (D_TnormTB * 4);
D_ScoreCasualTC := (D_AlphaTC * 6) + (D_TnormTC * 4);

D_ScoreImmTA := (D_AlphaTA * 4) + (D_TnormTA * 6);
D_ScoreImmTB := (D_AlphaTB * 4) + (D_TnormTB * 6);
D_ScoreImmTC := (D_AlphaTC * 4) + (D_TnormTC * 6);

D_EligCasualTA := SEL(M1, D_ScoreCasualTA, -1);
D_EligCasualTB := SEL(M2, D_ScoreCasualTB, -1);
D_EligCasualTC := SEL(M3, D_ScoreCasualTC, -1);

D_MaxID := 1;
D_MaxScore := D_EligCasualTA;
IF D_EligCasualTB > D_MaxScore THEN
    D_MaxScore := D_EligCasualTB;
    D_MaxID := 2;
END_IF;
IF D_EligCasualTC > D_MaxScore THEN
    D_MaxScore := D_EligCasualTC;
    D_MaxID := 3;
END_IF;

CASE D_MaxID OF
    1: D_RankedTankTemp := D_ThetaTA;
    2: D_RankedTankTemp := D_ThetaTB;
    3: D_RankedTankTemp := D_ThetaTC;
END_CASE;
```

4.6 Need-Array Arbiter

```
M_D_T5 := M_P1Demand;
M_D_TA := (M1 AND M_P2Demand) OR (NOT M1 AND (D_ThetaTA < 360));
M_D_TB := (M2 AND M_P2Demand) OR (NOT M2 AND (D_ThetaTB < 360));
M_D_TC := (M3 AND M_P2Demand) OR (NOT M3 AND (D_ThetaTC < 360));

D_NeedCount := 0;
IF M_D_T5 THEN D_NeedCount := D_NeedCount + 1; END_IF;
IF M_D_TA THEN D_NeedCount := D_NeedCount + 1; END_IF;
IF M_D_TB THEN D_NeedCount := D_NeedCount + 1; END_IF;
IF M_D_TC THEN D_NeedCount := D_NeedCount + 1; END_IF;

M_Serve_T5 := M_D_T5;
M_Serve_TA := M_D_TA;
M_Serve_TB := M_D_TB;
```

```
M_Serve_TC := M_D_TC;

IF (D_NeedCount = 4) THEN
    IF M1 THEN
        IF D_ScoreImmTB < D_ScoreImmTC THEN M_Serve_TB := FALSE;
        ELSE M_Serve_TC := FALSE;
        END_IF;
    ELSIF M2 THEN
        IF D_ScoreImmTA < D_ScoreImmTC THEN M_Serve_TA := FALSE;
        ELSE M_Serve_TC := FALSE;
        END_IF;
    ELSIF M3 THEN
        IF D_ScoreImmTA < D_ScoreImmTB THEN M_Serve_TA := FALSE;
        ELSE M_Serve_TB := FALSE;
        END_IF;
    ELSIF M0 THEN
        IF (D_ScoreImmTA <= D_ScoreImmTB) AND (D_ScoreImmTA <= D_ScoreImmTC)
↪ THEN
            M_Serve_TA := FALSE;
        ELSIF (D_ScoreImmTB <= D_ScoreImmTA) AND (D_ScoreImmTB <= D_ScoreImmTC
↪ ) THEN
            M_Serve_TB := FALSE;
        ELSE
            M_Serve_TC := FALSE;
        END_IF;
    END_IF;
END_IF;

D_ServeCount := 0;
IF M_Serve_T5 THEN D_ServeCount := D_ServeCount + 1; END_IF;
IF M_Serve_TA THEN D_ServeCount := D_ServeCount + 1; END_IF;
IF M_Serve_TB THEN D_ServeCount := D_ServeCount + 1; END_IF;
IF M_Serve_TC THEN D_ServeCount := D_ServeCount + 1; END_IF;

IF M2_SystemOK AND M3_AutoEnabled THEN
    Y6  := (D_ServeCount = 1);
    Y2  := (D_ServeCount = 2);
    Y17 := (D_ServeCount = 3);
END_IF;

Y11 := M_Serve_T5;
Y12 := M_Serve_TA;
Y13 := M_Serve_TB;
Y14 := M_Serve_TC;
```

4.7 Fault Detection

```
IF M10_Refresh THEN
    D_DeltaThetaTA := D_ThetaTA - D_ThetaTA_Prev;
    D_ThetaTA_Prev := D_ThetaTA;

    D_MaxHeatRateTA := (5011 * 100) / D_MassTA / 2000;
    D_MaxCoolRateTA := (668 * (D_ThetaTA - 181)) / D_MassTA / 2000;

    M_FaultHeatingTA := (D_DeltaThetaTA > D_MaxHeatRateTA);
    M_FaultCoolingTA := (D_DeltaThetaTA < 0) AND
                        (ABS(D_DeltaThetaTA) > D_MaxCoolRateTA);

    D_DeltaMassTA := D_MassTA - D_MassTA_Prev;
    D_MassTA_Prev := D_MassTA;
END_IF;
(* repeat identically for TB, TC *)

M_TAActiveTransfer := M_ServeTA_AODD OR M_ServeX OR M_ServeY OR M_ServeZ;
M_FaultLevelTA := (NOT M_TAActiveTransfer) AND (D_DeltaMassTA <> 0);

M_HighLimitTA := (D_ThetaTA > 750);
M_HighLimitTB := (D_ThetaTB > 750);
M_HighLimitTC := (D_ThetaTC > 750);

IF M_HighLimitTA OR M_HighLimitTB OR M_HighLimitTC THEN
    M_Alarm := TRUE;
    Y7  := FALSE;
    Y10 := FALSE;
    Y20 := FALSE;
END_IF;
```

5 Conclusion

1. $E_{\mathbf{total}}$ **is now actually computed**, not just consumed – closing the largest gap in the previous version of this document. The units error in E_5/E_{C^*}'s loss terms (Watts added to Joules with no time conversion) is fixed by introducing the planning window explicitly.

2. **Two real, unconfirmed decisions were made to get here**: the 5-minute planning window (proposed over the 1-hour figure considered during development, on staleness grounds), and the T5-only fast override (a sketched direction, not a complete per-duty implementation). Both are flagged inline and in the Nomenclature/ID tables, not buried.

3. D_Mp **and the Y7/Y10/Y20 gating are both now resolved** – m_p is T5's own computed remaining capacity, and the auxiliary elements now correctly drop under E-STOP.

4. **The Condition-3 taper in Dynamic Power Evaluation is confirmed correct as originally written** – it operates on *measured* water temperature as an independent safety ceiling, not on the dynamic target, and needed no change.

5. Every other open item from the previous version (Y21 asymmetry in oil distribution, tie-break on equal scores, E-STOP reset source, mode power-up default) remains open and unchanged.

6. **A proposed, unverified V/Z-indexed alternative** (Section 3.24) collapses the E_{total} and Fault Detection TA/TB/TC blocks from ∼54 to ∼22 named D registers using `FOR`/`NEXT` and a single Z0 index register – the single largest lever available against the D0–D127 general-use budget, but contingent on FOR/NEXT and Z-indexing support on the actual target hardware (unconfirmed) and on re-laying-out several currently name-addressed registers as contiguous arrays (not yet done elsewhere in this document).

Prepared by **Sambridda Ranabhat** · Mechatronics Engineering Pvt. Ltd. (MEPL), Kathmandu, Nepal

Based on the *VCH Sizing Framework, Second Edition*

DOI: 10.5281/zenodo.21009246